

特征值与奇异值分解实验

目标与可检查结果

本实验用小矩阵把手算、几何与 NumPy 对齐。你将计算实对称矩阵的特征分解，检验完整与薄 SVD 的形状，比较不同截断秩的误差，并主动构造“方向变了但答案没错”与“条件数有限但参数不唯一”的例子。

无需微积分，无需外部数据、账户、GPU 或付费接口。直接先修第十一讲。应能辨认矩阵行列数、转置、内积和矩阵乘法；不必先读懂整个脚本才能运行，但每一步需要解释输出为何符合数学条件。

本包含 experiment.ipynb、experiment.py 与 data 文件夹。Notebook 是逐步教学版本，脚本负责复用检查；data/main_matrix.csv 保存三通道矩阵，data/synthetic_field.csv 保存 32×32 原创强度场。请保持相对位置，不把 Notebook 单独移走。

一 启动环境并从头执行

如果已经能运行前面几讲，可继续使用已有环境。本实验实际验证环境为 Python 3.12.14、NumPy 2.3.5。若要与记录尽量一致，可以在 Anaconda Prompt 或终端执行下面三行；它们是可选的新环境安装步骤，需要网络，安装后的实验本身完全离线。

```
conda create -n ml012 python=3.12 numpy=2.3.5 matplotlib jupyterlab
conda activate ml012
jupyter lab
```

若指定版本因你的平台或频道不可用，不要随意从陌生网站下载二进制文件。使用官方安装方式已有的 NumPy 版本也可以尝试，但要实际版本记在结果里，不能将其他版本的输出冒充本讲验收环境。

也可以在 Anaconda Navigator 中启动 JupyterLab，进入保存本实验包的文件夹，再打开 experiment.ipynb。在菜单选择重启内核并运行全部单元，常见名称是“Restart Kernel and Run All Cells”。每段代码左侧的执行编号应按顺序出现，不能只挑后面的单元运行而依赖上次残留变量。

第一格会打印 Python 和 NumPy 版本，并显示所需文件是否存在。若文件检查为 False，先修正当前文件夹或恢复数据文件。不要把读取不到数据改成直接填预期答案。若提示找不到 NumPy，请检查 Notebook 选的内核是否就是已安装 NumPy 的环境。

本包已在新 Python 进程中的真实 ipykernel InProcessKernel 按顺序执行并保存输出。验证覆盖 Python 代码、依赖导入、读取 CSV 和数学断言；没有验证浏览器界面操作、外进程 socket 传输，也没有在你的本机 Anaconda 安装上测试。这些限制不改变已运行的数值检查，但应与“所有平台都已测试”区分。

二 先在纸上完成两项预测

在运行特征分解格之前，写下

$$B = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}.$$

计算 $(2 - \lambda)^2 - 1$ 并因式分解，找到 1、3 两个特征值。对每个值分别解 $(B - \lambda I)v = 0$ ，把一个非零向量归一化。不要只写两个数就算完成。

接着计算三通道矩阵

$$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ 1 & -1 \end{pmatrix}$$

的 $A^T A$ 。你应得到 $\begin{pmatrix} 6 & 3 \\ 3 & 6 \end{pmatrix}$ ，特征值为 9、3，因而 A 的奇异值是 3、 $\sqrt{3}$ 。注意不能把 9、3 直接称为 A 的奇异值。

先写下预期的两个单位输入方向，再用 Av_i/σ_i 算两个输出方向。讲义给出的答案是 $u_1 = (1, 1, 0)^T/\sqrt{2}$ 与 $u_2 = (1, -1, 2)^T/\sqrt{6}$ 。核验每列长 1，并核验内积为零。

三 检查特征分解而非死记向量符号

Notebook 调用 `np.linalg.eigh(B)`。返回的第一个数组是按升序排列的特征值，第二个矩阵按列放对应的单位特征向量。正文通常先讲最大值，因此叙述顺序与软件默认顺序不同。

运行后核对三个量： $B @ Q - Q * \text{eigenvalues}$ 应接近零； $Q.T @ Q - \text{np.eye}(2)$ 应接近零； $B - (Q * \text{eigenvalues}) @ Q.T$ 应接近零。第一项检查每个特征方程，第二项检查正交性，第三项检查完整重构。

符号不同是合法的。例如软件若给出 $(-1, -1)^T/\sqrt{2}$ ，它和 $(1, 1)^T/\sqrt{2}$ 是同一特征直线。仅凭截图不同不能判错；相反，重构误差小也不能替代形状与条件检查。

实验包装器 `symmetric_eigh` 会先检查输入是有限实方阵并严格对称。输入只接受整数和浮点实数；布尔值、数字字符串、复数、object 数组及一般对象叶子会在转成浮点数之前被拒绝，混合列表也按原始元素检查。NumPy 的 `eigh` 使用指定三角区域，本身不会为你报告上下三角不一致。尝试对 `[[1, 2], [0, 1]]` 调用包装器，预期是明确的 `ValueError`。Notebook 在 `try` 与 `except` 中捕获这次预期错误，所以整本仍能顺序运行。

四 分清返回数组的尺寸

先调用 `np.linalg.svd(A, full_matrices=True)`。对于本例 3×2 的 A，完整 U 是 3×3 ，s 是长度 2 的一维数组，Vt 是 2×2 。为了矩阵连乘，要自己构造 3×2 的 Σ ，把 s 填在前两个对角位置。

再设置 `full_matrices=False`, U 变为 3×2 , s 仍长2, Vt 仍为 2×2 。此时 $(U * s) @ Vt$ 可以重构 A : 乘法 $*$ 利用一维 s 对 U 的每列乘对应数, $@$ 才是矩阵乘法。也可以写 $U @ np.diag(s) @ Vt$, 更直接对应公式。

不要再次给 Vt 转置后才乘回去。NumPy 已经返回转置后的右向量矩阵; 如果你要取单根右奇异向量 v_i , 才从 $Vt.T[:, i]$ 取列。完整分解的 U 不能直接乘 2×2 的对角阵, 因为内侧维数不匹配。

对照脚本的 `shape_report`: 一个 3×2 秩亏矩阵薄 U 仍为 3×2 , 不会自动变成 3×1 。选择数值秩阈值并删除很小的奇异方向, 是另一步带有判断的操作。

五 手算并验证一层近似

保留最大奇异值对应层:

$$A_1 = \begin{pmatrix} 1.5 & 1.5 \\ 1.5 & 1.5 \\ 0 & 0 \end{pmatrix}.$$

相减得到误差矩阵 $A - A_1 = \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \\ 1 & -1 \end{pmatrix}$ 。六项平方之和为 $4 \times 0.25 + 1 + 1 = 3$ 。因此

Frobenius 误差是 $\sqrt{3}$, 不是3; 平方误差才是3。谱误差也是 $\sqrt{3}$, 因为只丢了一层。

总平方和为12, 保留率为 $1 - 3/12 = 3/4$ 。用 `Fraction` 的单元会精确得到整数3与分数3/4, 而根号相关的数组乘法需要浮点容差。请在实验笔记中保留这两种验证的区别。

函数 `truncated_svd(A, k)` 同时给出实际误差与理论误差。依次检查 $k=0, 1, 2$ 。 $k=0$ 应返回同尺寸零矩阵; $k=2$ 应重构 A , 仅有浮点舍入残差。尝试 $k=-1, k=3$ 或 $k=0.5$ 会被拒绝。函数并没有把错误参数静默改成合法整数。

六 故意改变符号与重复方向

将 U 第一列和 Vt 第一行同时乘 -1 , 再重构 A , 结果应保持不变。只改变 U 第一列时, 结果应不同, 本例 Frobenius 差约为6。用同一组数据并排比较这两个操作, 才能理解为什么“向量换号没关系”必须附带“左右成对”这个条件。

重复值实验使用 $D = \text{diag}(2, 2, 0.5)$ 。在前两个方向上选一个二维正交变换, 并同时作用在左右两侧。重构仍是 D , 但单根向量可以明显改变。比较变换前后的前两列投影矩阵, 它们的差应约为零。

再观察 $2I_2$ 的两种秩1近似: $\text{diag}(2, 0)$ 与 $\text{diag}(0, 2)$ 。它们不同, 却有同样的最优误差2。这是明确反例, 不需要用很多随机样本猜测最优解是否唯一。

七 查看合成强度图与误差曲线

数据场大小为 32×32 ，理论非零奇异值是 $[16, 8, 4, 2, 0.5, 0.25]$ 。其余计算出的值应在舍入噪声尺度。Notebook 会用固定0到1的色标显示原图及不同秩重构，不对每张图单独拉伸对比度。

记录三个结果：秩1保留约75.22%的平方和，秩2约94.03%，秩4约99.91%。秩1图只剩常量背景，尽管比例已不低。解释时要写“平方和能量”，不要写成“保留了75%的物体”“保留了75%的预测能力”。

误差曲线同时画出实际重构与理论值。Frobenius 理论误差为剩余奇异值平方和再开方；谱误差为第一个丢弃的奇异值。 $k=6$ 时理论误差按已知精确秩为零，而浮点重构可能留下约 10^{-14} 的误差。对完整浮点 SVD 的尾部数值计算，理论尾和本身也可能是这个小尺度。

本实验一般断言使用绝对容差 10^{-11} 和相对容差 10^{-11} ；截断误差比较的绝对容差再乘 $\max(1, \sigma_1)$ ，避免较大矩阵尺度被固定绝对阈值误判。这些容差适合本包的小型 float64 样例，不是对任何单位、规模和噪声的通用承诺。

八 检查条件数与 Gram 路线

先检查 $C = \text{diag}(1, 2^{-j})$ ， j 取0、10、20、30。程序应得到 $\kappa_2(C) = 2^j$ 、 $\kappa_2(C^T C) = 2^{2j}$ 。这是特别清楚的对角验证，但一般结论来自讲义证明，不能由四个例子推广。

然后对 $G = \begin{pmatrix} 1 & 1 \\ 0 & 2^{-j} \end{pmatrix}$ 比较直接 SVD 与先形成 $G^T G$ 的特征值再开方。实际环境在 $j=30$ 时，直接 SVD 的最小奇异值约 6.59×10^{-10} ，Gram 路线给出0。小方向在平方和舍入过程中丢失了，不是原矩阵在精确数学上突然少了一维。

不要先把所有 Gram 特征值取绝对值来“修复”：如果出现微小负数，先报告它、矩阵尺度和形成过程。取绝对值会把数值问题伪装成合法正值。代码记录原始特征值，出现负值时不进行实数开平方。

最后运行宽矩阵 $W = [I_2 \ 0]$ 。它的 cond 是1，但 $W(0, 0, 1)^T = 0$ 。在记录中写明：返回列表的奇异值比为1，输入零空间维数为1，因此参数仍不唯一。不能只抄一个条件数然后结论写“稳定且唯一”。

九 更换损失会改变最优答案

加权反例使用 $A = \text{diag}(3, 1)$ ，右下角误差的权重为100。无权 SVD 截断保留3，普通平方误差最小，但加权误差为100；另一个秩1矩阵保留1，加权误差只有9。

Notebook 会计算两个目标值。你的说明应包括矩阵、权重、候选秩和损失表达式，不能只说“某算法失败”。普通 SVD 正确解决了无权问题，换了损失后原来的最优性定理不适用。

十 验收与排错

最后运行完整检查。预期显示27组边界检查通过，同时完成已知谱、重构、正交、符号、重复值子空间、正定类别、低秩非唯一、截断误差和条件数核验。27是边界分组数，不是全部断言数量。

脚本也可独立运行：在实验目录打开终端，执行 `python experiment.py`。结果保存在相对路径 `outputs/experiment_report.json` 与 `outputs/truncation_errors.csv`。无需手工修改这些输出文件；改变实验参数后应重新生成，并保留改动说明。

若断言失败，先核对 NumPy 版本、数据是否改动、变量是否被后面的交互实验覆盖，再重启内核从头运行。不要为了得到绿色状态就扩大容差而不解释原因。若符号不同但正交、特征方程和重构都通过，应记录这种合法差异，无需强行改成答案截图。

提交一页实验记录，包含手算的 B 与 A 、至少两个截断秩的理论和实际误差、一个非唯一性检查、宽矩阵解释、加权反例，以及实际环境与未测试范围。把“程序在这些案例通过”和“定理对所有满足条件的矩阵成立”分别写清楚，即完成本实验。